

Stock Market Portfolio Tracker

Code Explanation and Setup Report

Generated for the Java Swing + JDBC + MySQL project in C:/Users/adris/Desktop/DB Clg
proj/stock-market-portfolio-tracker.

This report explains how to set up and run the project, how the architecture fits together, how data flows through the application, and what each source file does.

1. Setup Status

- Java is installed: javac 25.0.3 was detected.
- The project compiles successfully into the out directory using compile.ps1.
- The MySQL command-line client was not found on PATH, so database import could not be completed from this environment.
- DBConnection.java was updated to read DB_URL, DB_USER, and DB_PASSWORD from environment variables so credentials do not need to be hardcoded.

Compile command:

```
powershell -ExecutionPolicy Bypass -File .\compile.ps1
```

Run command after MySQL is installed/configured:

```
$env:DB_USER='root'  
$env:DB_PASSWORD='your_mysql_password'  
powershell -ExecutionPolicy Bypass -File .\run.ps1
```

2. Database Setup

Install MySQL 8.0 or newer, then run schema.sql in MySQL Workbench or CLI. The script creates the stock_tracker database and inserts demo data. Demo login: demo@tracker.com / password123.

If mysql is available on PATH, the import command is:

```
mysql -u root -p < schema.sql
```

Tables

Table	Purpose
User	Stores registered users and SHA-256 password hashes.
Portfolio	Stores portfolios owned by users.
Stock	Stores ticker symbols and company names.
Holding	Stores current portfolio positions and average buy prices.
Transaction	Stores BUY/SELL trade history.

Market_Data	Stores dated OHLCV market data; latest close is used as current price.
Anomaly	Stores detected unusual stock events and severity.
Alert	Stores user-facing messages linked to anomalies.

3. Architecture

The project follows a simple MVC-style structure. Model classes represent database records. DAO classes contain SQL and business persistence operations. View classes build Swing screens and call DAOs when the user clicks buttons or changes selections. DBConnection centralizes the JDBC connection. Main launches the first screen.

Layer	Package	Responsibility
Entry	com.tracker.app	Starts the Swing app.
Config	com.tracker.config	Creates/reuses the database connection.
Model	com.tracker.model	Simple data containers used by DAO and UI layers.
DAO	com.tracker.dao	SQL queries, inserts, updates, authentication, and trade logic.
View	com.tracker.view	Swing windows, dialogs, tables, forms, buttons, validation, and navigation.

4. Runtime Flow

- Main starts the program and shows LoginFrame.
- LoginFrame validates email/password and calls UserDAO.authenticate.
- On success, DashboardFrame opens with the authenticated User object.
- DashboardFrame loads portfolios from PortfolioDAO. Selecting a portfolio triggers refreshDashboard.
- refreshDashboard loads holdings, recent transactions, and alerts through HoldingDAO, TransactionDAO, and AlertDAO.
- New Trade opens TransactionDialog. A BUY recalculates weighted average price. A SELL validates share quantity before updating holdings.
- Market_Data is not fetched live from the internet. It is seeded in schema.sql and read as historical/local sample data.

5. File Inventory

File	Lines	Role
README.md	98	Project overview and manual run instructions. It identifies the stack as Java Swing, JDBC, and MySQL, lists the main user features, and documents the original compile/run commands.

schema.sql	179	Creates the stock_tracker database, eight normalized tables, foreign keys, and seed data. It is the data foundation for login, portfolios, holdings, trades, market prices, anomalies, and alerts.
compile.ps1	6	Local helper added for setup. It recursively finds every Java source file under src, compiles them with javac, and writes class files into out.
run.ps1	7	Local helper added for setup. It compiles when out is missing, then launches com.tracker.app.Main with both out and the vendored MySQL Connector/J JAR on the runtime classpath.
src/com/tracker/app/Main.java	26	The application entry point. It sets the cross-platform Swing look and feel, then uses SwingUtilities.invokeLater so the LoginFrame is created on the Swing Event Dispatch Thread.
src/com/tracker/config/DBConnection.java	73	Singleton JDBC connection manager. It loads com.mysql.cj.jdbc.Driver, opens a MySQL connection, and reconnects if the connection is null or closed. It now reads DB_URL, DB_USER, and DB_PASSWORD environment variables, falling back to localhost stock_tracker, root, and blank password.
src/com/tracker/model/User.java	42	Plain Java object for the User table. It stores user_id, name, email, and password_hash, with constructors, getters/setters, and a toString for debugging.
src/com/tracker/model/Portfolio.java	35	Plain Java object for a user's portfolio. The toString returns the portfolio name, which lets Swing list and combo-box components display readable names automatically.
src/com/tracker/model/Stock.java	35	Plain Java object for listed stocks. It stores stock_id, ticker, and company_name. Its toString combines ticker and company name, which is used directly in the trade dialog stock selector.
src/com/tracker/model/Holding.java	67	Plain Java object for a portfolio holding. It stores database fields plus display-only joined fields such as ticker, companyName, and currentPrice. getProfitLoss computes (currentPrice - avgBuyPrice) * quantity.
src/com/tracker/model/Transaction.java	64	Plain Java object for BUY/SELL transaction history. It stores transaction id, portfolio id, stock id, type, quantity, price, timestamp, plus a display-only ticker field populated by joins.
src/com/tracker/model/Anomaly.java	47	Plain Java object for the Anomaly table. It represents unusual stock events with stock id, date, type, and numeric severity.

src/com/tracker/model/Alert.java	55	Plain Java object for alert messages linked to anomalies. It also carries joined display fields: ticker, anomaly type, severity, and date.
src/com/tracker/dao/UserDAO.java	96	Handles registration and authentication. registerUser hashes the raw password with SHA-256 and inserts the user. authenticate hashes the typed password and searches for a matching email/hash row. PreparedStatement protects these queries from SQL injection.
src/com/tracker/dao/PortfolioDAO.java	70	Handles portfolio persistence. getPortfoliosByUser loads the current user's portfolios for the sidebar. createPortfolio inserts a new row for the logged-in user.
src/com/tracker/dao/StockDAO.java	69	Handles stock lookup. getAllStocks returns stocks alphabetically for the trade selector. getLatestClosePrice queries Market_Data by date descending and returns the newest closing price.
src/com/tracker/dao/HoldingDAO.java	150	Handles holding reads and writes. getHoldingsByPortfolio joins Holding and Stock and uses a subquery for latest market price. findHolding finds a portfolio/stock pair. insertHolding, updateHolding, and deleteHolding support trade execution.
src/com/tracker/dao/TransactionDAO.java	149	Implements the trading rules. executeBuy logs a BUY and creates or updates holdings with weighted average price. executeSell validates enough shares, logs a SELL, and reduces or deletes the holding. getRecentTransactions loads the latest 50 trades.
src/com/tracker/dao/AlertDAO.java	59	Loads relevant anomaly alerts for the selected portfolio. It joins Alert, Anomaly, Stock, and Holding so users only see alerts for stocks they own, sorted by severity and date.
src/com/tracker/view/LoginFrame.java	215	Swing login screen. It builds a dark themed card with email/password fields, validates non-empty input, calls UserDAO.authenticate, opens DashboardFrame on success, and opens RegisterFrame from the signup link.
src/com/tracker/view/RegisterFrame.java	224	Swing registration screen. It validates name, email shape, password length, and password confirmation, calls UserDAO.registerUser, shows success/error status, and returns to LoginFrame.
src/com/tracker/view/DashboardFrame.java	563	Main logged-in UI. It owns the selected user and portfolio, builds the header/sidebar/holdings/transactions/alerts layout, loads portfolios, refreshes tables, computes total value and profit/loss, creates portfolios, opens TransactionDialog, and renders severity-colored alert cards.

src/com/tracker/view/TransactionDialog.java	271	Modal trade form. It loads stocks, auto-fills latest market price, lets the user choose BUY or SELL, validates quantity and price, calls TransactionDAO, and reports success or failure with JOptionPane.
---	-----	---

6. File-by-File Explanation

README.md

Size: 98 lines, 2735 characters.

Project overview and manual run instructions. It identifies the stack as Java Swing, JDBC, and MySQL, lists the main user features, and documents the original compile/run commands.

schema.sql

Size: 179 lines, 7342 characters.

Creates the stock_tracker database, eight normalized tables, foreign keys, and seed data. It is the data foundation for login, portfolios, holdings, trades, market prices, anomalies, and alerts.

compile.ps1

Size: 6 lines, 202 characters.

Local helper added for setup. It recursively finds every Java source file under src, compiles them with javac, and writes class files into out.

run.ps1

Size: 7 lines, 156 characters.

Local helper added for setup. It compiles when out is missing, then launches com.tracker.app.Main with both out and the vendored MySQL Connector/J JAR on the runtime classpath.

src/com/tracker/app/Main.java

Size: 26 lines, 810 characters.

The application entry point. It sets the cross-platform Swing look and feel, then uses SwingUtilities.invokeLater so the LoginFrame is created on the Swing Event Dispatch Thread.

src/com/tracker/config/DBConnection.java

Size: 73 lines, 2616 characters.

Singleton JDBC connection manager. It loads com.mysql.cj.jdbc.Driver, opens a MySQL connection, and reconnects if the connection is null or closed. It now reads DB_URL, DB_USER, and DB_PASSWORD environment variables, falling back to localhost stock_tracker, root, and blank password.

src/com/tracker/model/User.java

Size: 42 lines, 1206 characters.

Plain Java object for the User table. It stores user_id, name, email, and password_hash, with constructors, getters/setters, and a toString for debugging.

src/com/tracker/model/Portfolio.java

Size: 35 lines, 957 characters.

Plain Java object for a user's portfolio. The toString returns the portfolio name, which lets Swing list and combo-box components display readable names automatically.

[src/com/tracker/model/Stock.java](#)

Size: 35 lines, 940 characters.

Plain Java object for listed stocks. It stores stock_id, ticker, and company_name. Its toString combines ticker and company name, which is used directly in the trade dialog stock selector.

[src/com/tracker/model/Holding.java](#)

Size: 67 lines, 2298 characters.

Plain Java object for a portfolio holding. It stores database fields plus display-only joined fields such as ticker, companyName, and currentPrice. getProfitLoss computes (currentPrice - avgBuyPrice) * quantity.

[src/com/tracker/model/Transaction.java](#)

Size: 64 lines, 2001 characters.

Plain Java object for BUY/SELL transaction history. It stores transaction id, portfolio id, stock id, type, quantity, price, timestamp, plus a display-only ticker field populated by joins.

[src/com/tracker/model/Anomaly.java](#)

Size: 47 lines, 1364 characters.

Plain Java object for the Anomaly table. It represents unusual stock events with stock id, date, type, and numeric severity.

[src/com/tracker/model/Alert.java](#)

Size: 55 lines, 1715 characters.

Plain Java object for alert messages linked to anomalies. It also carries joined display fields: ticker, anomaly type, severity, and date.

[src/com/tracker/dao/UserDAO.java](#)

Size: 96 lines, 3338 characters.

Handles registration and authentication. registerUser hashes the raw password with SHA-256 and inserts the user. authenticate hashes the typed password and searches for a matching email/hash row. PreparedStatement protects these queries from SQL injection.

[src/com/tracker/dao/PortfolioDAO.java](#)

Size: 70 lines, 2171 characters.

Handles portfolio persistence. getPortfoliosByUser loads the current user's portfolios for the sidebar. createPortfolio inserts a new row for the logged-in user.

[src/com/tracker/dao/StockDAO.java](#)

Size: 69 lines, 2186 characters.

Handles stock lookup. getAllStocks returns stocks alphabetically for the trade selector. getLatestClosePrice queries Market_Data by date descending and returns the newest closing price.

[src/com/tracker/dao/HoldingDAO.java](#)

Size: 150 lines, 5485 characters.

Handles holding reads and writes. `getHoldingsByPortfolio` joins Holding and Stock and uses a subquery for latest market price. `findHolding` finds a portfolio/stock pair. `insertHolding`, `updateHolding`, and `deleteHolding` support trade execution.

[src/com/tracker/dao/TransactionDAO.java](#)

Size: 149 lines, 5814 characters.

Implements the trading rules. `executeBuy` logs a BUY and creates or updates holdings with weighted average price. `executeSell` validates enough shares, logs a SELL, and reduces or deletes the holding. `getRecentTransactions` loads the latest 50 trades.

[src/com/tracker/dao/AlertDAO.java](#)

Size: 59 lines, 2321 characters.

Loads relevant anomaly alerts for the selected portfolio. It joins Alert, Anomaly, Stock, and Holding so users only see alerts for stocks they own, sorted by severity and date.

[src/com/tracker/view/LoginFrame.java](#)

Size: 215 lines, 8484 characters.

Swing login screen. It builds a dark themed card with email/password fields, validates non-empty input, calls `UserDAO.authenticate`, opens `DashboardFrame` on success, and opens `RegisterFrame` from the signup link.

[src/com/tracker/view/RegisterFrame.java](#)

Size: 224 lines, 8766 characters.

Swing registration screen. It validates name, email shape, password length, and password confirmation, calls `UserDAO.registerUser`, shows success/error status, and returns to `LoginFrame`.

[src/com/tracker/view/DashboardFrame.java](#)

Size: 563 lines, 23080 characters.

Main logged-in UI. It owns the selected user and portfolio, builds the header/sidebar/holdings/transactions/alerts layout, loads portfolios, refreshes tables, computes total value and profit/loss, creates portfolios, opens `TransactionDialog`, and renders severity-colored alert cards.

[src/com/tracker/view/TransactionDialog.java](#)

Size: 271 lines, 10698 characters.

Modal trade form. It loads stocks, auto-fills latest market price, lets the user choose BUY or SELL, validates quantity and price, calls `TransactionDAO`, and reports success or failure with `JOptionPane`.

7. Important Implementation Details

Authentication

Passwords are hashed with SHA-256 in `UserDAO` before storage or comparison. This is better than storing plaintext, but a production-grade system should use salted adaptive hashing such as `bcrypt`, `scrypt`, or `Argon2`.

Trade Logic

BUY records are inserted into Transaction first. If the holding already exists, the average buy price is recalculated by weighted average: $((oldQty * oldAvg) + (newQty * newPrice)) / (oldQty + newQty)$. SELL records require an existing holding with enough quantity, then reduce or delete the holding.

Portfolio Valuation

DashboardFrame computes total value as $currentPrice * quantity$ for every holding. Profit/loss is computed per holding by `Holding.getProfitLoss` and summed for the selected portfolio.

Alert Filtering

AlertDAO only shows alerts for stocks present in the selected portfolio by joining Holding with Stock, Anomaly, and Alert. This makes alerts contextual instead of global.

UI Threading

Main correctly creates the first Swing frame on the Event Dispatch Thread. DAO calls are currently made directly from Swing event handlers, which is acceptable for a college-scale demo but can freeze the UI if the database is slow. SwingWorker would be a good future improvement.

8. Suggested Improvements

- Add a connection test screen or startup error dialog when MySQL is unavailable.
- Use DECIMAL instead of FLOAT for money values in schema.sql.
- Wrap BUY/SELL transaction logging and holding updates in a single SQL transaction for atomicity.
- Add UNIQUE(portfolio_id, stock_id) to Holding to prevent duplicate holdings for the same stock.
- Add salt and adaptive password hashing for stronger security.
- Add automated tests for DAO trade logic using a test database.